TEST PLAN FOR

MARKETSAFE

Note that you can refine your testing plan as the project development goes. Keep the change log as follow:

ChangeLog

Version	Change Date	By	Description
1.0	March 1st, 2026	Michael	Sprint 2 features added, Cover feature: 1,2,5
1.1	March 5th, 2026	Fidelio	added more details for the roles, more details for the CI/CD workflow description

1 Introduction

1.1 Scope

Scope defines the features, functional or non-functional requirements of the software that will be tested.

The 3 features we have completed and have been tested for this sprint are Account Management System, Marketplace Listings and Search. For each feature we have implemented both unit and integration testing.

The Account Management System contains 4 functional requirements we need to test. These are: Account Registation, Email Verification, Login and Logout and View Profile. The Account

Registration System provides secure access to the platform by only allowing users with a registered University of Manitoba email to create an account. When creating an account we verify that an email is a UofM domain and the email is not linked to an account already and we ensure the password fulfills our safety requirements. If these checks pass users are given a unique user ID. Once registered and verified, users can log in and log out of the system using JWT authentication. During login, valid credentials must generate a signed JWT containing the user ID and expiration, while invalid credentials must return proper error responses. The issued JWT must be included in any requests to allow access to protected routes. Logout must invalidate the user's authenticated session on the client side by removing the stored JWT and ensure that requests made without a valid token are denied access. Any unauthenticated or improperly authenticated user attempting to access protected pages must be redirected to the login page and prevented from accessing restricted resources. A logged in user can access their own profile if holding a valid JWT.

Marketplace Listings contains 3 functional requirements: Create a Listing, Delete a Listing and Comment on a Listing. When creating a listing, we verify all required information is included before creating the listing and adding the listing to the database. Once a listing is created, it should be visible in the marketplace. Users should be able to delete their own listing and not others. Once a listing is deleted it should be removed from the database and the marketplace. Users should be able to comment on posted listings in the marketplace. Comments posted by a valid logged in user should be displayed under the listing post. Users should be able to see their own and other users' comments on listings.

Search consists of 3 functional requirements: Search Listings, Filter Listings by Tags and Sort Listings. To search for a listing, users can input keywords that relate to the listings they wish to view. Listings matching the chosen keywords will be displayed to the user. When the user changes or removes a keyword, the listings displayed to the user will update accordingly. Users can filter by tags by selecting one or more listing attribute from a given list. Listings displayed to the user will change accordingly to only show listings fulfilling these attributes. If a user removes a tag, the listings displayed to the user will update accordingly. Lastly a user can sort the displayed listings by selecting one of the given sort criteria. When a criterion is selected, the list of displayed items will reorganize into the users desired configuration. If the user changes this sort criteria, the list will update its organization of listings accordingly.

1.2 Roles and Responsibilities

Detail description of the Roles and responsibilities of different team members:

- **QA Analyst**
- **Test Manager & Architect**
- **Backend developer**
- **Frontend developer**
- **Backend Architect**
- **DB Designer & Architect**

Name	Net ID	GitHub username	Role
Fidelio Ciandy	Ciandyf	FidelioC	Backend Developer Frontend Developer QA Analyst
Farah Hegazi	Hegazif	Farahheg	Frontend Developer QA Analyst
Michael King	Kingm4	MichaelKing1619	Backend Developer Test Manager QA Analyst
Mohamed, Mohamed Youssef Mohamed	Mohammym	lMollMol	Backend Developer & Architect DB Designer & Architect Test Manager & Architect QA Analyst
Thomas Awad	Awadt	Tomtom4343, michel00angelo	Backend Developer QA Analyst

Role details:

- QA Analyst
  - Developed and executed unit and integration tests for both frontend and backend components.
  - Identified defects, reported issues, and ensured new features met functional and quality requirements.
- Test Manager & Architect
  - Defined the project's testing strategy, including test coverage goals and testing frameworks.
  - Coordinated testing efforts across the team and ensured CI pipelines enforced test quality standards.
- Backend developer
  - Implemented backend APIs and business logic using FastAPI.
  - Managed database interactions and ensured reliable data processing for core features such as accounts, listings, and comments.
- Frontend developer
  - Developed the user interface using Angular and integrated frontend components with backend APIs.
  - Implemented responsive layouts and user workflows for listing management and account interactions
- Backend Architect
  - Designed the backend architecture, including API structure, service layers, and system integration.
  - Ensured scalability, maintainability, and adherence to clean architecture principles..

- DB Designer & Architect
    - Designed the relational database schema for entities such as accounts, listings, and comments.
    - Defined relationships, constraints, and indexing strategies to ensure efficient and reliable data storage.

2 Test Methodology

2.1 Automated regression testng

2.1.1 Test Levels

Test levels define the Types of Testing to be executed on the Application Under Test (AUT). In this course, unit testing, integraiton tesing, acceptance testing, regression testing, and laod testing are mandatory.

Requirements:

- **For unit testing, at least 10 unit tests for EACH core feature to cover the code related to each core feature**
- **For integration testing, at least 10 in total to cover core features.**
- **Acceptance testing for each core feature. Let's use end-user test for this. You can ask real end-user or your team members to go through each user story and see if the requrements are meet.**
- **For regression testing, need to execute all above unit tests + integration tests you have for each commit pushed to main branch.**

Test Level	Scope & Requirement	Methodology (How will you do this?)
Unit Testing	**[~150] tests per core feature.** Total: [451] tests.	We use python '**unittest**' to run unit tests and use Mock and Magic Mock to mock dependencies and verfiy logic in isolation.
Integration Testing	**[129] tests total** covering interactions between features.	Using 'unittest' to run tests. Integration tests are ran against a test database hosted in a Docker container. In each integration test class we set up a database session and ensure tables exist and restes the tables to ensure a clean state for all test classes. MySQL DAO implementations are created allowing us to check the systems real database interactions.

Test Level	Scope & Requirement	Methodology (How will you do this?)
Acceptance Testing	**End-user testing** for every user story.	Team members and selected end users will perform manual walkthroughs of the system based on user story acceptance criteria. For every feature, testers will follow realisic scenarios through the system UI to verify that the functionality fulfills the user requirements. During these feature walkthroughs, testers will validate expected outcomes, proper system feedback, proper authorization and general system behaviour for each task in a walkthrough.
Regression Testing	Unit + Integration tests executed on **every push to main branch**.	We configured a GitHub Actions CI pipeline to automatically run our unit, integration, and frontend tests on each pull request. All CI checks must pass before a merge is allowed.

2.1.2 CI/CD Regression Workflow

Frontend CI

- Triggered automatically on pushes and pull requests to the production and main branches.
- Checks out the repository and sets up a Node.js 20 environment using GitHub Actions.
- Installs frontend dependencies using npm ci to ensure reproducible builds.
- Runs ESLint to enforce code quality and style consistency.
- Executes Angular unit tests in a headless Chrome environment.

Backend CI

- Triggered automatically on pushes and pull requests to the production and main branches.
- Sets up a Python 3.11 environment and installs project dependencies using pip.
- Performs a Python syntax compilation check (compileall) to detect early code errors.
- Executes backend **unit tests** using the Python unittest framework while collecting coverage with coverage.py.
- Runs **integration tests** and appends results to the same coverage report.
- Generates coverage reports in both terminal and HTML formats for test coverage analysis.

General Workflow

- Runs two parallel CI jobs (frontend and backend) to validate both parts of the system.
- Ensures that code changes pass linting, testing, and build verification before being merged into the main branches.

2.2 Mutation Testing (Test effectiveness)

Skip for 2nd sprint.

2.3 Load Testing

Skip for 2nd sprint.

3 Terms/Acronyms

Make a mention of any terms or acronyms used in the project

TERM/ACRONYM	DEFINITION
API	Application Program Interface
AUT	Application Under Test
DAO	Domain Access Object
UI	User Interface
QA	Quality Assurance
JWT	JSON Web Token